6.8610 Quiz 2 Cheatsheet

Question cues: bottleneck first; watch absolute words (*always*, *only*, *guarantees*, *equivalent*); reject answers by missing property; say quality *and* systems constraint; simplify toy objectives before interpreting them.

Post-Training Flow

$$\text{pretrain} \to \text{mid-train} \to \text{SFT} \to \text{prefs} \to \text{RL}$$

Mid-train

- Continue next-token prediction on smaller, higher-value data: code, books, technical text, long-context/tool traces.
- Shorter ctx can be cheaper early; expand ctx later once basic priors exist.
- Later data matters more: better to forget early boilerplate than late high-value data.

SFT objective $\theta =$ *params,* $\mathcal{D} = \{(x, y)\}$ *demo pairs,* $x =$ *prompt,* $y =$ *assistant response.*

$$\max_{\theta} \sum_{(x,y)\in\mathcal{D}} \log p_\theta(y \mid x)$$

- Usually mask loss on prompt/user tokens; supervise mainly assistant tokens.
- Teaches instruction following, chat format, refusals, response structure.
- Extracts latent skills; does not replace pretraining.

SFT failure modes

- Label bottleneck.
- One gold answer when many would work.
- Can teach style of competence more than calibrated truth.
- "I don't know" is hard from demos alone.

Preference Learning

Setup $x =$ *prompt,* $y_w/y_\ell =$ *winner/loser,* $p_\theta =$ *policy,* $p_0 =$ *reference,* $\sigma =$ *logistic.*

Bradley–Terry $r_\theta(x, y) =$ *learned scalar preference score.*

$$P(y_w \succ y_\ell \mid x) = \sigma(r(x, y_w) - r(x, y_\ell))$$
$$\mathcal{L}_{\text{BT}} = -\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_\ell))$$

DPO shorthand

$$\Delta_\theta = \log \frac{p_\theta(y_w \mid x)}{p_0(y_w \mid x)} - \log \frac{p_\theta(y_\ell \mid x)}{p_0(y_\ell \mid x)}$$
$$\mathcal{L}_{\text{DPO}} = -\log \sigma(\lambda \Delta_\theta)$$

- Uses LM probs relative to reference; no separate reward model required.
- Reference model limits drift while satisfying winner/loser constraints.
- Pairwise labels are often easier than writing ideal completions.

Reward parameterization behind DPO $\lambda =$ *preference scale,* $Z(x) =$ *prompt-wise partition function.*

$$r_\theta(x, y) = \lambda \log \frac{p_\theta(y \mid x)}{p_0(y \mid x)} + \lambda \log Z(x)$$

If SLiC fits in memory *Here* $\Delta > 0$ *is a margin, not the DPO* Δ_θ *above.*

$$\max(0, \Delta + \log p_\theta(y_\ell \mid x) - \log p_\theta(y_w \mid x))$$

Keep straight

- SFT = demonstrations.
- DPO = preferences + reference.
- RL = optimize reward on fresh model samples.

RLHF Basics

Reward-regularized objective $x =$ *prompt,* $y =$ *completion,* $p_0 =$ *reference policy,* $\lambda =$ *KL weight.*

$$J(\theta) = \mathbb{E}_x \mathbb{E}_{y\sim p_\theta(\cdot|x)}[r(x, y)] - \lambda\, \text{KL}(p_\theta \| p_0)$$

Reward model $\phi =$ *reward-model params,* $h_\phi(x, y) =$ *features,* $w =$ *linear head.*

$$r_\phi(x, y) = w^\top h_\phi(x, y)$$
$$\max_{\phi} \log \sigma(r_\phi(x, y_w) - r_\phi(x, y_\ell))$$

Vanilla policy gradient $r(x, y) =$ *scalar reward.*

$$\nabla_\theta \mathbb{E}_{y\sim p_\theta}[r] = \mathbb{E}_{y\sim p_\theta}\left[\nabla_\theta \log p_\theta(y \mid x)\, r(x, y)\right]$$

Value / advantage

$$V(x) = \mathbb{E}_{y\sim p_\theta}[r(x, y)], \qquad A(x, y) = r(x, y) - V(x)$$

- Baseline reduces variance; does *not* change expected gradient.
- Vanilla PG is unbiased, high variance, on-policy.
- Big RL story: learn reward from scarce labels, then use compute on new samples.

KL-folding approx. $r'(x, y) =$ *reward after folding the reference-model KL term into the scalar reward.*

$$r'(x, y) = r(x, y) - \log p_\theta(y \mid x) + \log p_0(y \mid x)$$

Tiny cue

- Equal SFT probs, rewards 1/0, $\beta =$ KL weight $= 1 \Rightarrow$ better response gets prob $e/(e + 1)$.

PPO / GRPO

Importance ratio $p_{\theta_{\text{old}}} =$ *old policy,* $A =$ *advantage,* $\epsilon =$ *clip radius.*

$$\rho(x, y) = \frac{p_\theta(y \mid x)}{p_{\theta_{\text{old}}}(y \mid x)}, \qquad \bar{\rho} = \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon)$$

PPO objective

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_{y\sim p_{\theta_{\text{old}}}}\left[\min(\rho A, \bar{\rho} A)\right]$$

- Sample reuse via importance sampling.
- Advantage weighting.
- Clipping stabilizes policy updates.

GRPO $r_i =$ *rewards for samples* i *from one prompt,* $\mu, \sigma =$ *group mean/std.*

$$\mu = \text{mean}(r_i), \qquad \sigma = \text{std}(r_i), \qquad A_i = \frac{r_i - \mu}{\sigma}$$

- Same PPO-style clipping, but no learned value model.
- Group normalization is convenient when many samples come from one prompt.

Fast compare

- **SFT:** imitate demos.
- **DPO:** fit preferences against reference.
- **PPO:** practical RL workhorse.
- **GRPO:** simpler RL variant with within-group normalization.

What usually matters

- supervision quality
- reward credit
- stability
- whether fresh model samples are needed

6.8610 Quiz 2 Cheatsheet

Retriever tradeoffs

Method	Interaction	Precompute	Query cost	Best use
XE	full q-d	none	$O(N)$ TF calls	rerank
DE	1 vec/doc	docs offline	$O(1)$ TF + $O(N)$ dots	first stage
Late int.	token-level	doc toks offline	small q TF + ANN	middle ground
ANN / IVF	prune candidates	index offline	sublinear-ish shortlist	scale

Neural Retrieval

Dual-encoder score $q = query,\ d = doc,\ d^+ = positive\ doc,\ d_i^- = negative\ docs.$

$$s(q,d) = f(q)^\top g(d)$$
$$s^+ = s(q,d^+), \qquad s_i^- = s(q,d_i^-)$$

InfoNCE $\tau = temperature,\ s^+ = positive\ score,\ s_i^- = negative\ scores.$

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(s^+/\tau)}{\exp(s^+/\tau) + \sum_i \exp(s_i^-/\tau)}$$

Late interaction / ColBERT $E_{q_i}, E_{d_j} = query/doc\ token\ embeddings.$

$$S(q,d) = \sum_i \max_j E_{q_i}^\top E_{d_j}$$

Retrieval reminders

- Cross-encoders are great rerankers, bad first-stage retrievers.
- Hard negatives matter more than easy random negatives.
- Single-vector dense retrieval often fails on OOD generalization.
- Late interaction keeps precompute *and* token-level matching.
- ANN still works: high score needs at least one close token vector.

RAG

Closed-book answer from params only.
Open-book retrieve docs and answer from evidence.
RAG factorization $q = query,\ a = answer,\ d = doc,\ \mathcal{D} = corpus,\ p_\eta = retriever,\ p_\theta = reader.$

$$p(a \mid q) = \sum_{d \in \mathcal{D}} p_\eta(d \mid q)\, p_\theta(a \mid q, d)$$

practice: approximate over top-K

Reader choice

- extractive = span; generative = decode over docs

Retriever supervision

- Often only (q, a), not gold docs; answer-containing doc is only a proxy.
- Search relevance and answer usefulness are related, not identical.

Credit assignment

- Right doc in top-K + wrong answer $\Rightarrow$ reader/evidence-use failure.
- Latent doc marginalization separates retriever success from reader failure; junk top-K can still push memorization.

FiD

- encode passages separately; decoder attends across all

Agents

Inference scaling

- Parallel: pass@k, best-of-n, self-consistency. Sequential: longer CoT / "Wait" forcing.

Pass@1 vs pass@k

- pass@1 = one rollout succeeds.
- pass@k = at least one of k rollouts succeeds.

Agent loop $s_t = state,\ a_t = action,\ o_{t+1} = observation.$

$$s_t \to a_t \to o_{t+1} \to s_{t+1}$$

Tool use

- emit tool call, execute externally, append observation, continue generation

ReAct vs CodeAct

- **ReAct:** interleave reasoning and actions.
- **CodeAct:** act by writing code; more general, more dangerous.

Long horizon

- context rot + stateful environments
- compaction keeps active context small while preserving task state
- mention environment interaction and cost, not just answer quality

Decoding

Greedy trap

- local argmax $\neq$ best full sequence

Best-of-n $p = base\ policy,\ q = best\text{-}of\text{-}n\ induced\ policy,\ n = number\ of\ samples.$

$$\mathrm{KL}(q \| p) \le \log n$$

- sample n full outputs, keep best by reward/verifier; strong baseline
- search error $\neq$ model error; large policy shift needs exponential n

Speculative / Contrastive

Speculative $p_t = target\ next\text{-}token\ dist,\ q_t = draft\ dist,\ \alpha = accept\ prob,\ A = accept\ rate.$

$$\alpha(x) = \min\left(1, \frac{p_t(x)}{q_t(x)}\right), \qquad A = 1 - \tfrac{1}{2}\|p_t - q_t\|_1$$

- draft q_t proposes, target p_t verifies; final token law still p_t
- best when draft is close to target; bigger draft can raise acceptance but lose speed

Contrastive $p_t^+ = expert,\ p_t^- = amateur,\ \lambda = contrast\ penalty.$

$$s_t(x) = \log p_t^+(x) - \lambda \log p_t^-(x)$$

$$\max_x s_t(x) \equiv \max_x \frac{p_t^+(x)}{(p_t^-(x))^\lambda}$$

- small λ: near expert decoding
- large λ: penalize generic amateur-favored tokens; too large hurts coherence

LM Programming

Natural-language signatures

- specify *what*, not model-specific *how*; e.g. `thread -> events, actions`

Three ingredients

- code / modular structure
- natural-language specs
- data / feedback

Compile intent

- prompts, weights (FT / RL), inference pipeline

Intervention map

- underspecified task $\Rightarrow$ modularize + prompt/program optimization
- missing knowledge $\Rightarrow$ retrieval, data, or weights
- prompt optimization cannot add knowledge; it complements FT / RL

Tokenization

Representation tradeoff

- byte: finite vocab, long seqs
- word: short seqs, brittle OOV/morphology
- subword: practical middle ground

Idealized objective $\Sigma = vocab,\ f = tokenizer/segmentation\ fn,\ V = target\ vocab\ size.$

$$\min_{\Sigma, f} \sum_i |f(x^{(i)})| \quad \text{s.t. } |\Sigma| = V$$

BPE

- start from bytes/chars; repeatedly merge most frequent adjacent pair
- store merge list; apply merges greedily at test time

Other families

- WordPiece: likelihood / PMI flavored
- Unigram LM: start big, prune
- SentencePiece: BPE / Unigram without whitespace assumptions

Keep straight

- pretokenization matters; compression $\neq$ downstream quality
- no universal winner; special chat tokens stay atomic